MOBILE ARCHITECTURE LAB 4 (MAL4) • MARTECH CAMPAIGN REVIEW & APPROVAL

Architecture Decision Records

Formal Architectural Decisions, Decision Drivers, Alternatives Evaluation, Trade-Off Analyses, and Requirements Reconciliation (ADR-001 through ADR-006)

Document ID:	MAL4-ADR-v1.0
Track:	MarTech — Campaign Review & Approval
Author / Role:	Architecture & Engineering Lead
Last Updated:	2026-11-01

Document Status:	ACCEPTED / REVIEWED
Primary Style:	Modular Monolith / Clean-Lite
Solution Design:	MAL4-HLD.md / MAL4-LLD.md
Reconciliation:	ADR-006 TRACKED

Executive Architecture Brief & Core Decision Governance

This document formalizes the complete Architecture Decision Records (ADRs) for the MAL4 Campaign Review & Approval platform. ADRs 001 through 005 establish the foundation of the current Solution Design (HLD/LLD): a Modular Monolith with Feature-First Clean-Architecture-lite layering, immutable full-snapshot campaign versioning, dual-check optimistic concurrency control, in-process domain events, and on-device role-based access control sized for a two-person team and hackathon timeline.

Every decision reflects explicit trade-offs: full Clean Architecture ceremony is pruned to preserve velocity while retaining domain isolation; distributed transaction machinery is rejected to fit a single-deployable mobile footprint; and persistence is isolated behind repository interfaces to maintain future evolution seams.

Architectural Divergence & Reconciliation Governance (ADR-006)

Explicit Conflict Disclosure: Inspection of the requirements baseline (01-problem-understanding, 04-workflows, 06-data-design) identified that it now specifies a 3-stage sequential approval pipeline (Creative → Compliance → Finance) with SHA-256 snapshot hashing, server-side relational enforcement, and a transactional outbox table. ADR-006 formally tracks this discrepancy rather than papering over it, outlining three defensible resolution options for team adoption.

ADR Status Summary Matrix

ADR ID	DECISION TITLE	STATUS	ARCHITECTURAL SCOPE & ROLE
ADR-001	Overall Architecture Style	ACCEPTED	Modular Monolith, Feature-First organization, Clean-lite layering, MVVM, Repository pattern
ADR-002	Campaign Versioning, Approval-Version Binding & OCC	ACCEPTED	Immutable snapshots, optimistic concurrency control, dual domain/repository conflict checks
ADR-003	In-Process Domain Events for Decoupling	ACCEPTED	Post-commit domain events, non-blocking asynchronous notification dispatch
ADR-004	Authorization Boundary & No-Backend Model	ACCEPTED (SOLUTION DESIGN)	On-device RBAC & Anti-Self-Approval; explicit disclosure of no-server boundary
ADR-005	Local-First Persistence Strategy	ACCEPTED (SOLUTION DESIGN)	Drift over SQLite; atomic transaction across campaign state and audit log
ADR-006	Multi-Stage Approval & Backend Reconciliation	PROPOSED — UNRESOLVED	Reconciliation between 6-state Solution Design and 3-stage server-backed requirements

Section 1: Architecture Decision Records — Index & Relationships

Architecture Decision Records — Index

> **Owner:** Architecture & Engineering Lead **Last Updated:** 2026-11-01

This index lists every ADR for the MAL4 Campaign Review & Approval project, its status, and its relationships to the other ADRs and to the Solution Design (`docs/02-solution-design/`).

Status Summary

ADR	TITLE	STATUS
ADR-001	Overall Architecture Style — Modular Monolith, Feature-First, Clean-Architecture-Lite	ACCEPTED
ADR-002	Campaign Versioning, Approval-Version Binding, and Optimistic Concurrency	ACCEPTED
ADR-003	In-Process Domain Events for Side-Effect Decoupling	ACCEPTED
ADR-004	Authorization Enforcement Boundary and No-Backend Deployment Model	ACCEPTED (SOLUTION DESIGN)
ADR-005	Local-First Persistence Strategy (Drift over SQLite)	ACCEPTED (SOLUTION DESIGN)
ADR-006	Reconciliation of Multi-Stage Sequential Approval and Server-Side Enforcement with the Current Solution Design	PROPOSED — UNRESOLVED

How to Read This Collection

ADR-001 through ADR-005 document decisions that are genuinely reflected in the current, reviewed Solution Design (`docs/02-solution-design/HLD/MAL4-HLD.md`, `docs/02-solution-design/LLD/MAL4-LLD.md`). They are marked ACCEPTED because that is an accurate description of what was decided and built into that design — not because every open question in the wider project is settled.

ADR-004 and ADR-005 carry a qualifier because they are the two decisions most directly implicated by ADR-006: inspection of the requirements baseline (`docs/01-problem-understanding/`, `docs/04-workflows/`, `docs/06-data-design/`) found that it now specifies a 3-stage sequential approval pipeline with server-side, database-backed enforcement — materially different from what ADR-002 through ADR-005 describe. This was discovered during preparation of this ADR collection and is **not** a pre-existing, already-resolved part of the project; it is recorded as ADR-006, with status PROPOSED, precisely because it has not been resolved, and because silently rewriting ADR-002/004/005 to match the newer requirements would misrepresent the HLD/LLD as containing a decision it does not implement.

Relationships Between ADRs

```
ADR-001 (architecture style)
   ├── establishes the layering ADR-002, ADR-003, ADR-004, ADR-005 operate within
   │
   ├── ADR-002 (data consistency: versioning, approval binding, optimistic concurrency)
   │       └── implemented via the persistence mechanism in ADR-005
   │
   ├── ADR-003 (event processing: in-process domain events)
   │       └── exists because of the no-backend decision in ADR-004
   │
   ├── ADR-004 (authorization boundary, no backend)
   │       └── directly drives ADR-005's local-first choice
   │
   └── ADR-005 (local-first persistence)

ADR-006 (unresolved reconciliation)
   └── contradicts ADR-002, ADR-003, ADR-004, and ADR-005 simultaneously —
       tracks the conflict between the Solution Design and the current
       requirements baseline; does not select a resolution
```

Scope and Ownership Note

This index and all ADRs in this folder are owned and maintained by the Architecture & Engineering Lead. The requirements documentation referenced in ADR-006 (`docs/01-problem-understanding/`, `docs/04-workflows/`, `docs/06-data-design/`) is owned by the Domain Research & Requirements Lead and has not been modified as part of producing this ADR collection.

Section 2: ADR-001 — Overall Architecture Style

ADR-001: Overall Architecture Style — Modular Monolith, Feature-First, Clean-Architecture-Lite

Status

ACCEPTED

Decision Date

2026-11-01

Context and Problem

The MAL4 Campaign Review & Approval mobile application needs an internal architecture that (a) keeps campaign lifecycle and approval invariants testable without a UI or database, (b) stays proportionate to a two-person team and a hackathon timeline, and (c) does not foreclose future evolution (a real backend, multi-stage approval — see ADR-006) by hard-coding assumptions throughout the codebase.

Six candidate concepts were on the table — Layered Architecture, Clean Architecture, Feature-First organization, MVVM, the Repository pattern, and Event-Driven Architecture. Treating these as five or six competing "pick one" options would be a category error: each answers a different architectural question (deployment topology, code organization, internal dependency direction, presentation pattern, and inter-component communication, respectively). The real decision is which combination to adopt, and at what level of rigor.

Decision Drivers

- Lifecycle and approval rules (campaign state transitions, version binding, self-review prevention) must be unit-testable in isolation — Testing & Observability is an explicitly weighted MAL4 evaluation criterion.
- Team size is two; any architecture with ceremony that doesn't pay for itself in testability or safety is a direct cost to delivery.
- No distributed infrastructure is justified at the confirmed scale (single mobile app, local-first).
- Persistence technology is not yet finalized (ADR-005) and the backend question is unresolved (ADR-006) — the internal architecture must not assume either answer.
- The solution must remain coherent if a feature boundary (e.g. multi-stage approval) is added later without a rewrite.

Considered Alternatives

ALTERNATIVE	DESCRIPTION
Classic Layered Architecture (global UI / business-logic / data-access layers)	Partition the whole codebase by technical layer type, shared across all features.
Full Clean Architecture	Strict four-ring architecture (entities, use cases, interface adapters, frameworks) with DTOs and mappers at every boundary.
Feature-First + Clean-Architecture-lite (**selected**)	One feature module (`campaign`), internally layered (domain/application/infrastructure/presentation) by dependency direction only, without full Clean Architecture's ring ceremony.
Full Event-Driven Architecture	All inter-component communication via published events on a broker or event bus.

Evaluation and Comparison

Scored against this project's actual constraints (two-person team, hackathon timeline, Flutter/Dart, 5–50 users / 10–200 active campaigns per the confirmed non-functional requirements):

CRITERION	CLASSIC LAYERED	FULL CLEAN ARCHITECTURE	FEATURE-FIRST + CLEAN-LITE (SELECTED)	FULL EVENT-DRIVEN
Separation of concerns	Medium — by technical type, not business capability	HIGH	HIGH	HIGH for decoupling, orthogonal to the others
Complexity	Low	HIGH	Low–Medium	HIGH if applied broadly
Testability	Medium — global layers tangle unrelated features	HIGH	HIGH	Medium — async flows are harder to test
Maintainability as the app grows	Low — unrelated features share layers	HIGH, at a ceremony cost	HIGH	Low if overused — hides control flow
Development speed for a 2-person team	HIGH	Low	HIGH	Low if applied broadly
Suitability for Flutter	Medium	Medium — common but often overbuilt	HIGH — standard modern Flutter practice	Low for a single mobile app

Selected Decision

Modular Monolith (single deployable, no services) + **Feature-First** code organization (one `campaign` feature) + **Clean-Architecture-lite** internal layering (domain → application ← infrastructure, presentation depends on application only — dependency rule enforced, full ring/DTO ceremony not adopted) + **MVVM** in the presentation layer (state notifiers as the view-model) + **Repository pattern** as the persistence dependency-inversion seam + **narrow, in-process domain events** for decoupling side effects only.

Rationale

Full Clean Architecture's ceremony (strict rings, mapper classes at every boundary) is disproportionate to one feature and two developers — the *principle* that is actually valuable (dependency inversion, a domain with zero outward dependencies) is retained without the ceremony that doesn't pay for itself at this scale. Feature-First avoids coupling the campaign feature to any future feature through shared global layers. MVVM is the natural shape for a Riverpod/Bloc-style notifier

regardless of this choice, and operates one level below it. Narrow in-process events are justified by a specific, named requirement (notification failures must never block or roll back a committed state transition) rather than adopted as a general pattern.

Positive Consequences

- The domain (`Campaign` aggregate and its invariants) has zero outward dependencies and is testable without Flutter, a database, or a network connection.
- Persistence technology remains swappable without touching domain or application code (dependency inversion via repository interfaces).
- The module boundary (`Review & Approval Workflow`) that would need to absorb multi-stage approval, if that becomes the adopted scope (see ADR-006), is already isolated from Campaign Management, Audit, and Notification modules.

Negative Consequences and Trade-offs

- Some boilerplate (use-case classes, repository interfaces, typed `Result` returns) versus directly calling a database from a widget — accepted because it is what buys the testability property above.
- "Clean-lite" risks boundary ambiguity without a strict reference implementation to point to; mitigated by writing explicit layer responsibility tables in the Solution Design rather than relying on convention alone.
- A Feature-First, single-feature codebase does not by itself prove the module boundaries are right for a second feature that doesn't exist yet — this is accepted as a reasonable bet given the project has exactly one feature in scope.

Risks and Mitigations

RISK	MITIGATION
Two developers interpret "Clean-lite" differently and the dependency rule erodes over time	Dependency-direction rules are stated explicitly per layer in the Solution Design, with a target lint rule to enforce `domain/` importing nothing from Flutter or the persistence library (not yet configured — no code exists yet)
The architecture is scoped to a single-approval, local-first assumption that the current requirements baseline no longer matches (see ADR-006)	This ADR records the style decision independent of that question — Feature-First + Clean-lite does not itself prevent the module from being extended to a multi-stage pipeline or a real backend; the reconciliation is tracked separately in ADR-006, not silently folded in here

Requirement Traceability

REQUIREMENT	HOW THIS DECISION ADDRESSES IT
Testing & Observability (MAL4 evaluation criterion)	Domain layer isolation is what makes lifecycle/approval logic unit-testable without a UI or database
NFR-015 (workload scalability, single-instance sufficient)	Modular Monolith matches the confirmed scale; no distributed infrastructure is introduced
Team-size constraint (2 engineers)	Feature-First + Clean-lite, not full Clean Architecture, keeps delivery cost proportionate

Assumptions and Open Risks

This decision assumes the project remains a single-feature mobile application for its hackathon deliverable scope. It does not resolve whether the requirements baseline's 3-stage approval pipeline and server-side enforcement model (ADR-006) will be adopted — that is a separate, currently unresolved decision.

Related ADRs and Design Sections

- Solution Design (HLD): `docs/02-solution-design/HLD/MAL4-HLD.md` §4.6 (Architecture Style and Rationale), §4.8 (Logical Architecture).
- Low-Level Design (LLD): `docs/02-solution-design/LLD/MAL4-LLD.md` §5.2 (module packaging).
- ADR-002 (data consistency), ADR-003 (event processing), ADR-004 (authorization boundary), ADR-005 (persistence) all operate inside the layering this ADR establishes.
- ADR-006 (multi-stage approval & server-side enforcement reconciliation) — unresolved; this ADR's decision is scoped to the current Solution Design, not the expanded requirements baseline.

Section 3: ADR-002 — Data Consistency, Versioning & Concurrency

ADR-002: Campaign Versioning, Approval-Version Binding, and Optimistic Concurrency

Status

ACCEPTED

Decision Date

2026-11-01

Context and Problem

A campaign proposal is edited, submitted, reviewed, possibly revised, and eventually scheduled. Three failure modes must be architecturally prevented, not merely discouraged:

1. **Post-approval drift** — a reviewer approves one version of a campaign, but a different, later-modified version is the one that gets scheduled.
2. **Stale approval** — a reviewer acts on a version of the campaign that is no longer current, without realizing it has changed underneath them.
3. **Lost updates** — two concurrent writers (two sessions, or a marketer and an in-flight review decision) overwrite each other's changes silently.

A decision was required on how campaign state is versioned, how an approval decision is bound to the exact content it evaluated, and how concurrent writes are detected and resolved.

Decision Drivers

- Approval decisions must be defensible — "which exact version was approved" must be an unambiguous, machine-checked fact, not a convention.
- The confirmed non-functional requirements set (durability, concurrent-edit safety, version-approval binding) are explicit and critical-priority.
- Team size and timeline rule out heavyweight distributed-transaction machinery.
- The approval model assumed at the time this decision was made is single-reviewer (see ADR-006 for the later, unresolved multi-stage requirement).

Considered Alternatives

Versioning

ALTERNATIVE	DESCRIPTION
In-place mutation	A campaign's current fields are edited directly; no separate version record exists.
Event-sourcing delta diffs	Each edit is stored as a diff; current state is reconstructed by replaying diffs.
Immutable full-snapshot versions (**selected**)	Each submission or revision creates a complete, immutable snapshot of campaign content; the aggregate's "current version" is a pointer, not the editable record itself.

Concurrency Control

ALTERNATIVE	DESCRIPTION
Pessimistic locking	A reviewer or marketer acquires an exclusive lock on the campaign row while evaluating or editing it.
Two-phase commit / distributed transaction coordination	A distributed transaction protocol coordinates writers across nodes.
Optimistic concurrency via a version token (**selected**)	Every write includes the version the writer believes is current; the write is rejected if the stored version has advanced.

Evaluation and Comparison

CRITERION	IN-PLACE MUTATION	EVENT-SOURCING DIFFS	IMMUTABLE SNAPSHOTS (SELECTED)
Prevents post-approval drift	No — nothing stops the "current" record from changing after approval	Yes, in principle	Yes — an approval is bound to a specific, permanently fixed snapshot
Implementation complexity	Low	HIGH (replay/projection logic, snapshot reconstruction)	Low–Medium
Storage cost	Lowest	Medium	HIGH er (full copies), accepted — payloads are small
Auditability	Weak — history is not naturally preserved	Strong	Strong — every version is a permanent, queryable record

CRITERION	PESSIMISTIC LOCKING	DISTRIBUTED 2PC	OPTIMISTIC CONCURRENCY (SELECTED)
Fit for a single-device, low-write-contention mobile app	Poor — holding a lock across a reviewer's "thinking time" is impractical	Not applicable — no distributed nodes exist to coordinate	Good
Operational complexity	Medium (lock timeout/deadlock handling)	HIGH	Low
User experience on conflict	Blocked until lock released	N/A	Immediate, explicit conflict signal; caller reloads and retries

Selected Decision

Immutable full-snapshot versioning — a `CampaignVersion` is created only at two trigger points (submission, revision-resubmission), never on every edit. **Optimistic concurrency** via a `currentVersion` integer token compared at every write (`CampaignRepository.save(campaign, expectedVersion)`), and a

separate, second check inside the domain aggregate (`approve` / `reject` / `requestChanges`) that compares the reviewer's believed-current version against the aggregate's actual current version before any `ApprovalRecord` is created — distinct from the repository-level write check, because the two answer different questions ("is this review stale" vs. "did someone else write first") and warrant different user-facing messages.

Rationale

Immutable snapshots make "which version was approved" a structural fact rather than a convention that can be silently violated. Optimistic concurrency fits the access pattern (infrequent conflicts, a single device, reviewers taking minutes-to-hours to decide) far better than pessimistic locking, which would have to hold a lock across exactly the kind of long "thinking time" this workflow involves.

Positive Consequences

- An `ApprovalRecord.versionReviewed` field, once written, can never retroactively apply to a different version — enforced by always deriving it from the aggregate's own current version, never from caller-supplied input.
- Concurrent-write conflicts fail loudly and immediately (a typed `ConcurrentModificationError` or `StaleReviewError`), never silently.
- No lock-timeout tuning, deadlock detection, or distributed coordinator is required.

Negative Consequences and Trade-offs

- A true race — two conflicting writes arriving within the same instant, before either has completed — is not fully closed by this mechanism alone; it only guarantees the *second* write to complete detects the conflict. A formal idempotency-key scheme for retried network requests is a related but distinct concern, not yet designed (see Risks below).
- Rejected writes require the caller to reload and resubmit — there is no automatic merge. This is a deliberate choice (silent merging of campaign fields is not safe), but it is a real UX cost under contention.
- Storing full snapshots rather than diffs costs more storage per version; accepted because campaign payloads are small and the simplicity/auditability gain outweighs it.

Risks and Mitigations

RISK	MITIGATION
No idempotency-key scheme exists yet for retried network/client requests (distinct from the version-conflict mechanism above)	Flagged explicitly as an open item in the Solution Design; not claimed as solved
The requirements baseline now describes a 3-stage sequential approval pipeline with per-stage approval records bound to a cryptographic snapshot hash (SHA-256) — a materially different binding model than the single `ApprovalRecord` this ADR describes	See ADR-006 — this is an unresolved reconciliation, not silently absorbed into this ADR

Requirement Traceability

REQUIREMENT	HOW THIS DECISION ADDRESSES IT
Approval state consistency, version-approval binding (critical-priority, non-functional requirements)	Immutable snapshots + version-token binding on `ApprovalRecord`
Concurrent-edit safety (high-priority, non-functional requirements)	Optimistic concurrency at the repository boundary
Campaign versioning and revision functional requirements	Two version-creation trigger points, never on plain edits
Material-change re-approval functional requirement	Any edit to an Approved campaign routes through an explicit invalidation step before the edit is applied, producing its own audit entry

Assumptions and Open Risks

This ADR assumes a single `ApprovalRecord` per version (single-reviewer model). The requirements baseline's multi-stage pipeline would require this data model to extend to per-stage approval records bound to the same snapshot — an extension, not a contradiction, of the immutable-snapshot principle, but not yet designed. See ADR-006.

Related ADRs and Design Sections

- Solution Design (HLD) §4.11 (Data Architecture), §4.14 (Reliability).
- Low-Level Design (LLD) §5.2.3–§5.2.4, §6.1–§6.4.
- ADR-001 (architecture style — domain layer owns these invariants).
- ADR-005 (persistence strategy — where the atomic state+audit commit is implemented).
- ADR-006 (multi-stage approval reconciliation — the open question of whether this single-`ApprovalRecord` model extends or is superseded).

Section 4: ADR-003 — In-Process Domain Events

ADR-003: In-Process Domain Events for Side-Effect Decoupling

Status

ACCEPTED

Decision Date

2026-11-01

Context and Problem

A successful campaign state transition (e.g. an approval) should trigger a notification to the affected marketer. Notification delivery depends on an external service and can fail, time out, or be slow. A decision was required on how to connect "the state transition happened" to "a notification should be attempted" without letting the second concern compromise the first.

Decision Drivers

- A confirmed, critical reliability requirement: a notification failure must never block or roll back an already-committed, authoritative state transition.
- The architecture has no backend and no message broker (see ADR-004) — any mechanism must work in a single process.
- Team size and timeline rule out operating external messaging infrastructure.

Considered Alternatives

ALTERNATIVE	DESCRIPTION
Synchronous inline dispatch	The use case calls the notification service directly, in the same call path as the state transition, before returning to the caller.
External message broker / queue	State transitions publish to a broker (e.g. a managed queue service); a separate consumer processes notifications.
In-process domain events (**selected**)	The use case emits a plain in-memory event object after its transaction commits; a subscriber (`NotificationGateway`) consumes it asynchronously within the same process.

Evaluation and Comparison

CRITERION	SYNCHRONOUS INLINE	EXTERNAL BROKER	IN-PROCESS DOMAIN EVENTS (SELECTED)
Isolates notification failure from the core transaction	No — a failure or timeout in the inline call can delay or fail the user-facing response	Yes	Yes
Operational footprint	None extra	A managed broker/queue to provision, configure, and monitor	None extra — plain in-process objects
Fit for a single-device, no-backend architecture	Workable but risky	Disproportionate — no distributed consumers exist to justify it	Good fit
Delivery guarantee sophistication	None beyond "it happened or it didn't"	Strong (broker-managed retries, dead-lettering)	Weaker — retry/backoff must be hand-built; acceptable at this scale

Selected Decision

Use cases emit a typed, in-process domain event (e.g. `CampaignApproved`) only **after** their database transaction has committed. `NotificationGateway` subscribes to these events and attempts delivery asynchronously; any failure is logged and retried internally, and is never surfaced back to the use case or the end user as a transaction failure.

Rationale

The ordering is the entire point: emitting the event after commit, not before or as part of the same transaction, is what structurally guarantees a notification failure cannot roll back state that has already been durably recorded. A broker is not justified — there is exactly one consumer (`NotificationGateway`) and no distributed producers/consumers to coordinate.

Positive Consequences

- The reliability requirement (notification failure never blocks the write path) is satisfied by construction, not by convention — the event literally does not exist until after the write has succeeded.
- No additional infrastructure to operate, monitor, or pay for.
- The event types form a natural, compiler-checked (sealed-class) catalog of "things that happen" in the domain, useful for future consumers beyond notifications (e.g. analytics) without touching use-case code.

Negative Consequences and Trade-offs

- Retry/backoff logic for failed notification delivery must be hand-built rather than inherited from broker infrastructure; the exact backoff policy is not yet finalized.
- If the application process terminates between commit and successful notification delivery, an in-flight notification may be lost — there is no durable outbox table in the current design. This is a known, stated limitation, not a hidden one.
- This mechanism only works because everything runs in one process; it does not generalize if a real backend is introduced (see ADR-004/ADR-006) without further design work.

Risks and Mitigations

RISK	MITIGATION
A notification can be lost if the process dies before delivery (no durable outbox)	Stated explicitly as an open implementation detail in the Solution Design; an outbox table is a candidate future addition, not yet designed
The requirements baseline describes a durable, database-backed Transactional Outbox pattern (an `outbox_events` table with status tracking and exponential backoff) — a stronger durability guarantee than this ADR's in-memory event bus provides	This is part of the broader backend/architecture reconciliation tracked in ADR-006, not silently adopted here

Requirement Traceability

REQUIREMENT	HOW THIS DECISION ADDRESSES IT
Partial-failure recovery (notification failure isolation — high-priority, non-functional requirement)	Event emission strictly after commit; failures absorbed entirely within `NotificationGateway`
Lifecycle notifications functional requirement	Domain events map 1:1 to the notification triggers the requirement specifies

Assumptions and Open Risks

This ADR assumes in-memory delivery loss on process termination is an acceptable risk for the current deliverable scope. If the requirements baseline's durable-outbox model (ADR-006) is adopted, this ADR would need to be superseded, not silently reinterpreted.

Related ADRs and Design Sections

- Solution Design (HLD) §4.10 (Interface and Communication Architecture), §4.14 (Reliability).
- Low-Level Design (LLD) §5.2.6 (Notification Management), Figure 3 (Error Recovery Sequence).
- ADR-001 (architecture style — this is the one sanctioned use of event-driven communication, narrowly scoped).
- ADR-004 (no-backend deployment boundary — why a broker was not considered viable).
- ADR-006 (the requirements baseline's durable outbox pattern is a stronger guarantee than this ADR provides — unresolved).

Section 5: ADR-004 — Authorization Boundary & Deployment Model

ADR-004: Authorization Enforcement Boundary and No-Backend Deployment Model

Status

ACCEPTED (SOLUTION DESIGN) — **see ADR-006 for an unresolved conflict with the requirements baseline**

Decision Date

2026-11-01

Context and Problem

Role-based access control (only a Marketer may create/edit/submit; only a Reviewer, and never the submitter, may approve/reject/request changes) must be enforced somewhere that cannot be bypassed by a modified or malicious client. The confirmed non-functional requirements state this in explicit client-server terms: authentication must reject unauthenticated requests, and "authorisation must be enforced server-side, not only in the client." No backend process exists or is planned for the current Solution Design. A decision was required on where enforcement actually happens, and how to be honest about what that does and does not guarantee.

Decision Drivers

- The requirement is phrased in literal server-side terms; the architecture has no server.
- A 2-person team and hackathon timeline make standing up and operating a real backend a significant, separate undertaking.
- No prior decision in this project (ADR-001 or otherwise) establishes that a backend exists — it must not be silently assumed.
- Judges and reviewers will reasonably ask how authorization can be trusted without a server.

Considered Alternatives

ALTERNATIVE	DESCRIPTION
Build a minimal real backend for the hackathon deliverable	Stand up an actual server process (even a thin one) that performs authentication/authorization independently of the mobile client.
On-device enforcement, presented as equivalent to server-side	Enforce checks in application/domain code on-device, without acknowledging the gap against the literal requirement.
On-device enforcement, gap stated explicitly (**selected**)	Enforce checks in application/domain code on-device; explicitly document that this does not satisfy a literal production security reading of "server-side," as an accepted, stated limitation.

Evaluation and Comparison

CRITERION	REAL BACKEND	ON-DEVICE, GAP HIDDEN	ON-DEVICE, GAP STATED (SELECTED)
Literally satisfies NFR-008/009	Yes	No (same as the left column — the gap doesn't disappear by not mentioning it)	No, and says so
Proportionate to a 2-person hackathon team	No — materially increases scope (hosting, auth infrastructure, API design, deployment)	Same engineering cost as the selected option	Same engineering cost — the only difference is honesty
Defensible under judge cross-examination	Strong, if actually built	Weak — an unstated gap discovered by a reviewer reads as an oversight	Stronger than "hidden" — demonstrates the team understands the boundary and made a conscious, reasoned trade-off
Prevents a modified client from bypassing checks that *are* enforced (self-review, role, version)	N/A (not selected)	Only as strong as the on-device checks, regardless of framing	Same technical strength; difference is only in disclosure

Selected Decision

Authorization and self-review checks are enforced in the **Application and Domain layers**, on-device, in the same process as the UI — never relying on the UI merely hiding a button. This is explicitly documented as satisfying the *intent* of the requirement (checks cannot be bypassed by skipping client-side validation alone) but **not** a literal production security reading of "server-side enforcement," because there is no separate process boundary between "client" and "server" for a modified client binary to be unable to cross.

Rationale

Given no backend exists in this deliverable, the only real choice is whether to state the resulting gap plainly or not — the technical strength of the enforcement is identical either way. Stating it plainly is the only choice consistent with this project's standard of not claiming guarantees that don't hold, and it is also the more defensible position in an architecture review: it demonstrates the team identified the exact boundary of what their design can and cannot guarantee, rather than having it discovered by a reviewer.

Positive Consequences

- Self-review prevention, role checks, and version-binding checks are enforced in domain code that a UI cannot be coerced into skipping — a materially stronger guarantee than UI-only validation, even without a server.
- The gap against the literal requirement is recorded once, precisely, and consistently, rather than being implicitly re-discovered (and possibly mischaracterized) in multiple places.
- The `AuthProvider` interface already exists as a seam; introducing a real Identity Provider behind it later does not require rewriting use cases.

Negative Consequences and Trade-offs

- A sufficiently motivated attacker with a modified client binary could, in principle, bypass on-device checks — there is no process boundary preventing it. This is the core accepted limitation.
- "Enforced in the domain layer" is not equivalent to "enforced server-side" for any production deployment with real adversarial users; this architecture, as currently designed, is not production-ready from a security standpoint without a backend.

Risks and Mitigations

RISK	MITIGATION
A judge or reviewer treats this as disqualifying rather than as a scoped, disclosed trade-off	The limitation is stated consistently across the Solution Design, this ADR, and the Architecture Defence package — never hidden, always paired with the specific architectural reason (team size, timeline, proportionality)
The requirements baseline explicitly and repeatedly assumes a server-side API (HTTP status codes, a `Campaign API Gateway`, database-level `UPDATE ... WHERE version = :expected_version` enforcement)	This is the central subject of ADR-006 — not resolved by this ADR, which documents only what the current Solution Design actually built

Requirement Traceability

REQUIREMENT	HOW THIS DECISION ADDRESSES IT
Authentication (must reject unauthenticated requests)	`AuthProvider.currentUser()` gate before any use case proceeds
Authorization enforcement, self-review prevention (must not rely on the client alone)	Checked in Application layer and defensively again in the Domain aggregate — never only in Presentation
"Server-side" enforcement (literal requirement)	**Not fully satisfied** — explicitly documented as the accepted gap this ADR records

Assumptions and Open Risks

This ADR assumes the hackathon deliverable's threat model does not require defending against a malicious, modified client — an assumption that would not hold in a production deployment. It assumes no backend will be introduced for the current deliverable; if that assumption changes (ADR-006), this ADR would be superseded, not silently reinterpreted.

Related ADRs and Design Sections

- Solution Design (HLD) §4.13 (Security Architecture), §4.15 (Deployment and Runtime Architecture — "The NFR-008/009 Gap, Stated Plainly").
- Low-Level Design (LLD) §5.2.1 (Authentication & Authorization module).
- ADR-001 (architecture style — the Application/Domain layering this decision relies on).
- ADR-005 (persistence strategy — the companion decision on where data lives).
- ADR-006 (multi-stage approval & server-side enforcement reconciliation) — the requirements baseline's server-side RBAC expectation is not met by this ADR and is tracked there as unresolved.

Section 6: ADR-005 — Persistence Strategy

ADR-005: Local-First Persistence Strategy (Drift over SQLite)

Status

ACCEPTED (SOLUTION DESIGN) — **see ADR-006 for an unresolved conflict with the requirements baseline**

Decision Date

2026-11-01

Context and Problem

Campaign state, version history, approval records, and an append-only audit trail must be durably persisted, with atomic commits across the state change and its audit entry. A persistence technology and topology (local-only vs. backend-synced) was required, consistent with the no-backend decision in ADR-004.

Decision Drivers

- Durability and transactional atomicity requirements (a state change and its audit entry must commit together or not at all).
- No backend exists for the current deliverable (ADR-004) — ruling out a remote database as the primary store.
- FR-015's requirement that audit entries be queryable by campaign, actor, and time range favors a relational query model.
- Team size and timeline.

Considered Alternatives

ALTERNATIVE	DESCRIPTION
Raw SQLite	Direct SQL against a local SQLite file, no type-safety layer.
Isar (NoSQL/object database)	Local object-store database, schema-less-leaning, optimized for object graphs rather than relational joins.
Backend API + managed database	A remote relational database (e.g. PostgreSQL) accessed via a backend API.
Drift over SQLite (**selected**)	A type-safe, code-generated query layer over local SQLite, with migrations and reactive streams.

Evaluation and Comparison

CRITERION	RAW SQLITE	ISAR	BACKEND API + DB	DRIFT OVER SQLITE (SELECTED)
Type safety at compile time	No	Partial	N/A (network boundary)	Yes
Fit for relational joins (campaign → versions → approvals → audit)	Workable, hand-written	Weaker — object-store model, not natively relational	Strong	Strong
Transactional atomicity for state+audit commit	Yes, manually	Yes	Yes, but across a network boundary	Yes, natively
Requires a backend to exist	No	No	Yes — contradicts ADR-004	No
Engineering cost for a 2-person team	Medium (more hand-written SQL, more defect risk)	Low	HIGH (hosting, API design, auth infra)	Low–Medium
Supports FR-015's query-by-campaign/actor/time-range requirement	Yes, manually indexed	Workable but less natural	Yes	Yes, with standard SQL indexes

Selected Decision

Local-first persistence using Drift over SQLite. The campaign-state write and its corresponding audit-entry write commit in a single local database transaction. No remote database or backend API is part of the current Solution Design.

Rationale

Given ADR-004 already rules out a backend for this deliverable, the backend-API alternative is not actually available without reopening that decision — it is listed here for completeness of comparison, not as a live option. Between the remaining local options, Drift's relational model fits the domain's actual shape (one campaign has many versions, each may have one approval record, every transition has an audit entry) better than an object-store, and its type safety and migration support reduce defect risk for a 2-person team without materially increasing build complexity over raw SQLite.

Positive Consequences

- Atomic state+audit commit is native to the chosen engine, not something the application has to coordinate by hand.
- Offline operation is the baseline mode, not a special case, because there is no network dependency in the core workflow.
- Drift's in-memory test database makes repository-level testing straightforward without a real file.

Negative Consequences and Trade-offs

- **No cross-device synchronization.** Two installations of the app, on two devices, have no mechanism to see each other's data. This is a hard limitation, stated explicitly, not an oversight.
- No backup/restore or disaster-recovery infrastructure exists at this layer; SQLite's WAL-mode durability covers on-device crash-safety only, not loss of the device itself.
- Multi-tenancy is not addressed (consistent with the project's single-tenant scope assumption).

Risks and Mitigations

RISK	MITIGATION
A stakeholder expects multi-device support from "persistence" without reading the stated limitation	Stated consistently across the Solution Design, this ADR, and the Architecture Defence package
The requirements baseline's data design specifies a relational backend schema (PostgreSQL-style, with table partitioning, an `outbox_events` table, and an `idempotency_records` table) that assumes a server process this ADR does not provide	Tracked in ADR-006 as an unresolved reconciliation — this ADR documents only the current Solution Design's local-first decision

Requirement Traceability

REQUIREMENT	HOW THIS DECISION ADDRESSES IT
Durability, atomic audit/state commit (critical-priority, non-functional requirements)	Single local transaction per use-case execution
Audit queryability by campaign/actor/time range (functional requirement)	Relational schema with standard indexes
No distributed infrastructure disproportionate to team/timeline	Local-only engine, no hosting or network dependency

Assumptions and Open Risks

This ADR assumes multi-device synchronization is out of scope for the current deliverable. If the requirements baseline's server-backed data design (ADR-006) is adopted instead, this ADR would be superseded, not silently reinterpreted — the physical schema itself remains the responsibility of the data-design documentation regardless of outcome.

Related ADRs and Design Sections

- Solution Design (HLD) §4.7 (Technology Architecture), §4.11 (Data Architecture), §4.15 (Deployment and Runtime Architecture).
- Low-Level Design (LLD) §5.2.3.G, §6.4 (Database Specifications, Concurrent Modification).
- ADR-002 (data consistency — the atomic commit this ADR implements).
- ADR-004 (no-backend decision — the direct driver of this choice).
- ADR-006 (the requirements baseline's server-backed data design — unresolved against this ADR).

Section 7: ADR-006 — Multi-Stage & Backend Reconciliation

ADR-006: Reconciliation of Multi-Stage Sequential Approval and Server-Side Enforcement with the Current Solution Design

Status

`PROPOSED` — UNRESOLVED CONFLICT

This ADR does not select an option. It exists to document a significant discrepancy discovered during architecture-decision documentation, so it is tracked explicitly rather than silently resolved in either direction.

Decision Date

Discovered 2026-11-01. Not yet resolved.

Context and Problem

The Solution Design (`docs/02-solution-design/`, HLD/LLD) implements: a **single-reviewer approval model** (one Reviewer role, one `ApprovalRecord` per version), **on-device, no-backend enforcement** (ADR-004), and **local-first persistence** with no server-backed schema (ADR-005).

During preparation of this ADR collection, inspection of `docs/01-problem-understanding/`, `docs/04-workflows/`, and `docs/06-data-design/` found that these documents — marked **"Document Status: Approved v1.0"** and with the relevant open questions marked "`RESOLVED`" — now specify a materially different architecture:

- A **3-stage sequential approval pipeline**: Stage 1 (Creative, role `ST-002a`) → Stage 2 (Compliance, role `ST-002b`) → Stage 3 (Finance, role `ST-002c`), each a distinct reviewer role with its own `StageApproval` record, all three required before a campaign reaches `Approved`.
- **Cryptographic snapshot binding**: a SHA-256 digest (`snapshot_hash`) of each campaign version, referenced by every stage approval record.
- **Server-side enforcement via a relational backend**: SQL `UPDATE ... WHERE version = :expected_version` optimistic-concurrency statements, HTTP status codes (`400`, `409`), a "Campaign API Gateway," and a full physical schema (`campaigns`, `campaign_snapshots`, `stage_approvals`, `review_decisions`, `audit_log`, `outbox_events`, `idempotency_records`, with partitioning and indexing strategy).
- A **Transactional Outbox pattern with a durable `outbox_events` table and exponential-backoff worker**, rather than this project's in-process event bus (ADR-003).
- An **HTTP-header-based idempotency-key scheme** (`Idempotency-Key` header, a dedicated `idempotency_records` table), which ADR-002 explicitly flagged as not yet designed in the current Solution Design.

This is traceable directly to the functional requirements, non-functional requirements, and stakeholders documentation (e.g. functional-requirements.md's OQ-009, originally "Open" with an "Initial recommendation: single-approval model," now marked "`RESOLVED`" in favor of the 3-stage pipeline). The requirements baseline changed after the Solution Design was built against its earlier version, and the two were never reconciled.

Why This Matters

This is not a minor terminology mismatch. It affects:

- **The campaign state model** — `PENDING REVIEW` is a single state in the current HLD/LLD; the requirements baseline subdivides it into three sequential sub-stages with their own role gates and transition rules.
- **The data model** — a single `ApprovalRecord` per version (ADR-002) versus up to three `StageApproval` records per version, each independently bound to a snapshot hash.
- **The deployment model** — no backend (ADR-004) versus an explicit, detailed server-side API and relational database design.
- **The concurrency mechanism** — in-process optimistic concurrency (ADR-002) versus the same principle expressed as SQL against a networked database, plus an HTTP-level idempotency-key scheme ADR-002 explicitly marks as not yet designed.
- **The side-effect mechanism** — an in-process event bus (ADR-003) versus a durable, database-backed Transactional Outbox.

Every ADR in this collection that touches these areas (ADR-002, ADR-003, ADR-004, ADR-005) has been written to accurately describe what the current Solution Design actually contains, with an explicit cross-reference to this ADR wherever the requirements baseline now diverges. None of them were silently rewritten to match the newer requirements, because doing so would misrepresent the HLD/LLD as already reflecting a decision it does not implement — and this task's scope does not permit modifying the HLD/LLD to make that true.

Options for Resolution (Not Yet Decided)

OPTION	DESCRIPTION	IMPLICATION
A. Extend the Solution Design to the 3-stage/backend model	Formally update `docs/02-solution-design/` (HLD/LLD) to adopt the sequential pipeline, cryptographic binding, and server-side architecture the requirements now specify.	Materially larger scope: a backend must be designed and built, not just the mobile client. `HIGH`est fidelity to current requirements; highest delivery risk for a 2-person team.
B. Scope the requirements back to match the delivered architecture	Formally revise the requirements baseline (with the Domain Research & Requirements Lead) to reduce to single-approval, local-first scope for the hackathon deliverable, with the 3-stage/backend model explicitly deferred as a documented future-evolution target.	Lowest delivery risk; requires the team to jointly agree the requirements update was premature or aspirational for this deliverable's actual scope.
C. Present a two-tier story: MVP architecture now, target architecture documented separately	Keep the current Solution Design as the implemented/defensible MVP, and keep the requirements baseline's 3-stage/backend model as an explicitly labeled "target production architecture" that the current design's extension points (module boundaries, repository interfaces) are built to support — without claiming it is built.	Requires discipline to keep the two framings clearly separated in every document and in the live defence; risks judges perceiving it as an excuse rather than a plan, if not presented carefully.

No option is selected here. This ADR's purpose is to make the conflict visible and force a conscious choice, not to make that choice unilaterally within the scope of this ADR/defence documentation task.

What Is Not In Scope for This ADR

- Modifying `docs/02-solution-design/` (HLD/LLD) — out of scope for the task that produced this ADR collection.

- Modifying `docs/01-problem-understanding/`, `docs/04-workflows/`, or `docs/06-data-design/` — teammate-owned.
- Silently picking Option A, B, or C and writing the rest of the ADR/defence package as if it had been decided.

Recommendation (Non-Binding)

Given the explicit team-size and timeline constraints that motivated ADR-001, ADR-004, and ADR-005, **Option C** is the most defensible starting position for the hackathon presentation itself — it allows the team to present a working, reasoned, honestly-scoped architecture while demonstrating awareness of the full target requirement. However, **Option A or B must still be formally chosen** before the project's documentation set can be considered internally consistent; Option C is a presentation strategy, not a substitute for that decision.

Requirement Traceability

REQUIREMENT SOURCE	STATUS RELATIVE TO CURRENT SOLUTION DESIGN
`01-problem-understanding/functional-requirements.md` OQ-009 ("RESOLVED": 3-stage pipeline)	Not implemented in the current HLD/LLD
`01-problem-understanding/non-functional-requirements.md` (server-side enforcement)	Acknowledged as unmet in ADR-004, not newly discovered here
`04-workflows/campaign-lifecycle.md`, `approval-workflow.md`, `revision-workflow.md`, `failure-recovery.md` (all "Approved v1.0")	Describe a server-backed, 3-stage architecture not reflected in the current Solution Design
`06-data-design/database-schema.md`, `domain-model.md`	Describe a relational backend schema with tables (`stage_approvals`, `outbox_events`, `idempotency_records`) that do not exist in the current persistence design (ADR-005)

Related ADRs and Design Sections

- ADR-001 (architecture style) — the module boundaries this ADR's resolution would extend or replace.
- ADR-002 (data consistency) — the single-`ApprovalRecord` model this would need to become multi-stage.
- ADR-003 (event processing) — the in-process event bus this would need to become a durable outbox.
- ADR-004 (authorization boundary) — the no-backend decision this directly contradicts.
- ADR-005 (persistence strategy) — the local-first decision this directly contradicts.
- Architecture Defence: `docs/08-architecture-defence/risks-and-limitations.md` and `docs/08-architecture-defence/anticipated-questions.md` present this conflict to judges honestly rather than concealing it.

Section 8: Architecture Decision Defence — Spoken-Ready Notes

Source: `docs/08-architecture-defence/adr-defence.md` • Rehearsed, cross-examination-ready talking points covering core decisions, alternatives rejected, technical trade-offs, limitations, and revisit criteria.

Architecture Decision Defence

Purpose: For each ADR, the condensed defence material — why this and not the alternative, what it costs, and under what conditions the team would revisit it. Full reasoning and comparison tables live in the ADRs themselves (`docs/03-architecture-decisions/`); this document is the rehearsed, spoken-ready version.

ADR-001 — Architecture Style

Decision summary: Modular Monolith, Feature-First organization, Clean-Architecture-lite internal layering, MVVM presentation, narrow in-process domain events.

Why this approach: The problem needs one property above all — business rules that can't be bypassed or silently violated — and that comes from isolating the domain, not from distributing the system. A single feature, two developers, and a hackathon timeline make every unit of architectural ceremony something that has to earn its cost in testability or safety.

Alternatives rejected: Full Clean Architecture (too much ring/mapper ceremony for one aggregate and two people); classic layered architecture (couples unrelated features through shared global layers — a problem this project doesn't have yet, but the style still has it); full event-driven architecture (no distributed producers/consumers exist to justify it).

Technical trade-offs: Some use-case and repository boilerplate versus calling the database straight from a widget — accepted because it is exactly what buys domain testability.

Limitations: "Clean-lite" is not a strict, named reference architecture — boundary discipline depends on the team, not a framework. No lint rule enforces the dependency direction yet.

Reconsider when: A second feature is added that would benefit from shared infrastructure across features, or the team grows past a size where informal boundary discipline holds.

ADR-002 — Data Consistency, Versioning, and Concurrency

Decision summary: Immutable full-snapshot campaign versions; approval bound to the exact version reviewed; optimistic concurrency via a version token, checked twice (once for staleness, once for write conflicts).

Why this approach: Post-approval drift — approving v1, scheduling v2 — is the single most damaging failure mode this system must prevent, and it must be prevented structurally, not by discipline. Optimistic concurrency fits a workflow where a reviewer might hold a campaign open for minutes or hours; pessimistic locks would freeze the system for that entire window.

Alternatives rejected: In-place mutation (no protection against drift at all); event-sourcing diffs (reconstruction complexity not justified at this scale); pessimistic locking (impractical hold times); two-phase commit (no distributed nodes to coordinate).

Technical trade-offs: No automatic merge on conflict — the second writer must reload and resubmit. Full snapshots cost more storage than diffs; accepted because campaign payloads are small.

Limitations: A true simultaneous race (not a retry-after-success) is not fully closed by this mechanism alone; a formal idempotency-key scheme is a related, separate, and not-yet-designed concern.

Reconsider when: Write contention becomes frequent enough that "reload and retry" becomes a real user-experience problem, or if the requirements baseline's per-stage approval model (ADR-006) is adopted, which would extend this to multiple approval records per version.

ADR-003 — Event Processing

Decision summary: In-process domain events, emitted only after a transaction commits, consumed asynchronously by a notification adapter whose failures are never surfaced back to the triggering operation.

Why this approach: The ordering — emit after commit, never before — is what structurally guarantees a notification outage cannot roll back an already-durable state change. No broker is justified: there is exactly one consumer in a single process.

Alternatives rejected: Synchronous inline notification dispatch (couples an external service's reliability to a core business transaction); external message broker (operational overhead with no distributed consumers to justify it).

Technical trade-offs: Retry/backoff logic is hand-built rather than broker-provided. No durable outbox table exists — an in-flight notification can be lost if the process terminates before delivery.

Limitations: This mechanism only works because everything runs in one process; it would need redesign if a real backend were introduced.

Reconsider when: The architecture gains a backend (ADR-006), at which point a durable, database-backed outbox becomes both possible and arguably necessary.

ADR-004 — Authorization Boundary and No-Backend Deployment

Decision summary: Role and self-review checks enforced in Application and Domain code, on-device; explicitly documented as not satisfying a literal server-side-enforcement reading of the requirements, because there is no server.

Why this approach: Given no backend exists in this deliverable, the technical strength of on-device enforcement is the same whether or not the gap is disclosed — the only real choice was whether to state it. Stating it is both more honest and, in an architecture review, more defensible than having a reviewer discover an unstated gap.

Alternatives rejected: Building a minimal real backend for the hackathon deliverable (materially larger scope, disproportionate to team size and timeline — a decision this ADR does not reopen, but that ADR-006 revisits as one explicit resolution path).

Technical trade-offs: A modified client binary could, in principle, bypass on-device checks. This is the central, named limitation of the architecture as it stands.

Limitations: Not production-ready from a security standpoint against adversarial users without a backend.

Reconsider when: The project moves toward production deployment, or the team resolves ADR-006 in favor of introducing a real backend.

ADR-005 — Persistence Strategy

Decision summary: Local-first persistence via Drift over SQLite; no remote database or backend API.

Why this approach: A direct consequence of ADR-004 — with no backend, a remote database was never a live option. Among local choices, Drift's relational model fits the domain's actual shape (campaign → versions → approvals → audit) better than an object-store, with meaningfully less defect risk than raw SQL for a two-person team.

Alternatives rejected: Raw SQLite (more hand-written SQL, more defect surface); Isar (weaker fit for relational joins); backend API + managed database (contradicts ADR-004).

Technical trade-offs: No cross-device synchronization — a hard limitation, not an oversight. No backup/restore infrastructure beyond on-device crash-safety.

Limitations: Single-tenant only; no disaster-recovery story beyond what SQLite's WAL mode provides.

Reconsider when: Multi-device support becomes a real requirement, or ADR-006 resolves toward a server-backed data model.

ADR-006 — The Unresolved Reconciliation

Decision summary: Not a decision — a documented conflict. The requirements baseline (updated after the Solution Design was built) now specifies a three-stage sequential approval pipeline with server-side, database-backed enforcement; the current Solution Design implements a single-approval, local-first, no-backend architecture. Neither has been changed to match the other.

Why this is presented, not hidden: Discovering and disclosing this during architecture-decision documentation is itself evidence of a working review process. The alternative — silently picking one side while writing the rest of the documentation as if it were already resolved — would be a materially worse failure than the discrepancy itself.

What it costs to acknowledge: It invites the hardest questions a judge can ask. `anticipated-questions.md` prepares direct, non-evasive answers for exactly that.

Reconsider when: Immediately — before final submission, ADR-006 recommends the team explicitly choose one of its three documented resolution paths (extend the architecture, re-scope the requirements, or present a clearly-separated MVP/target-state narrative).

Section 9: Requirements & Architecture Traceability Matrix

Source: `docs/07-quality-attributes/traceability.md` • Comprehensive end-to-end forward and backward traceability mapping requirements to ADRs, domain entities, and quality attributes.

Requirements, Architecture & Verification Traceability Matrix

Document Status: Approved v1.0 **Last Updated:** 2026-11-01 **Owner:** Domain Research & Requirements Lead **Traces to:** All sections in `docs/01-problem-understanding/`, `docs/04-workflows/`, `docs/06-data-design/`, and `docs/07-quality-attributes/`

1. Executive Summary & Traceability Governance

This Traceability Matrix serves as the single source of truth connecting business drivers, functional capabilities, quality attribute requirements, data design entities, workflows, and test verification methods for the **MAL4 MarTech Campaign Review & Approval Platform**.

It provides complete forward and backward traceability across the entire engineering lifecycle, directly addressing the MAL4 scoring dimensions:

- **Structure & boundaries (15%)**
- **Data & state management (15%)**
- **Testing & observability (15%)**
- **Failure & recovery handling (10%)**
- **Architecture defence (10%)**

2. Functional Requirements Traceability Matrix (`FR-001` to `FR-019`)

REQ ID	REQUIREMENT SUMMARY	STAKEHOLDERS	OWNING WORKFLOW	DOMAIN AGGREGATE / DB TABLE	QUALITY ATTRIBUTE	VERIFICATION METHOD
`FR-001`	Campaign Creation & Drafting	`ST-001` (Marketer)	`campaign-lifecycle.md` §4.1	`Campaign` / `campaigns`	Usability, Data Integrity	Unit test: `test_create_draft_campaign()`
`FR-002`	Campaign Parameter Editing	`ST-001` (Marketer)	`campaign-lifecycle.md` §2.1	`Campaign` / `campaigns`	Concurrency, OCC	Unit test: `test_concurrent_draft_edit()`
`FR-003`	Soft Archival / Deletion	`ST-001`, `ST-003`	`campaign-lifecycle.md` §2.1	`Campaign` / `campaigns`	Auditability	DB test: soft delete preserves audit log
`FR-004`	Submit Proposal for Review	`ST-001` (Marketer)	`user-journey-flow.mmd` Phase 1	`CampaignSnapshot` / `campaign_snapshots`	Security, Integrity	Integration test: `test_submit_locks_snapshot()`
`FR-005`	Withdraw Submission	`ST-001` (Marketer)	`approval-workflow.md` §4	`Campaign` / `campaigns`	Consistency	Integration test: `test_withdraw_before_review()`
`FR-006`	Multi-Stage Review Progression	`ST-002a`, `ST-002b`	`approval-workflow.md` §3	`StageApproval` / `stage_approvals`	Reliability, Security	Integration test: `test_stage1_to_stage2_advance()`
`FR-007`	Finance Sign-off / Full Approval	`ST-002c` (Finance)	`approval-workflow.md` §3.3	`StageApproval` / `campaigns`	Compliance, Auditability	Integration test: `test_stage3_final_approval()`
`FR-008`	Rejection Decision	`ST-002a/b/c`	`approval-workflow.md` §3.5	`ReviewDecision` / `review_decisions`	Governance	Integration test: `test_rejection_terminates_cycle()`
`FR-009`	Request Changes Decision	`ST-002a/b/c`	`approval-workflow.md` §3.4	`ReviewDecision` / `review_decisions`	Usability, Integrity	Integration test: `test_request_changes_attaches_notes()`
`FR-010`	Campaign Revision & Resubmission	`ST-001` (Marketer)	`revision-workflow.md` §2.1	`CampaignSnapshot` (***vN+1***)	Traceability, Integrity	Integration test: `test_resubmit_creates_v2_resets_stage1()`
`FR-011`	Feedback Visibility & Diffing	`ST-001`, `ST-002a/b/c`	`revision-workflow.md` §5	`ReviewDecision` / UI	Usability, Observability	UI test: structured field diff rendering
`FR-012`	Scheduling Gate Enforcement	`ST-001`, `ST-003`	`campaign-lifecycle.md` §4.8	`Campaign` / `campaigns`	Security, Governance	Integration test: `test_scheduling_gate_verifies_signatures()`
`FR-013`	Immutable Version Snapshots	`ST-001`, `ST-002a/b/c`	`revision-workflow.md` §3	`CampaignSnapshot` / `campaign_snapshots`	Cryptographic Integrity	Unit test: `test_snapshot_sha256_immutable()`
`FR-014`	Material Change Invalidation	`ST-001`, `ST-005`	`revision-workflow.md` §4	`Campaign` / `stage_approvals`	Compliance, SOX 404	Integration test: `test_budget_increase_invalidates_approval()`
`FR-015`	Immutable Audit Logging	`ST-003`, `ST-005`	`observability.md` §6	`audit_log` (Partitioned)	Auditability, Non-Repudiation	DB test: `INSERT` succeeds; `UPDATE` / `DELETE` blocked
`FR-016`	Approval History Timeline	`ST-002`, `ST-003`	`observability.md` §6	`audit_log` / Query Engine	Observability	API test: query full timeline by `campaign_id`
`FR-017`	Role-Based Access Control	All Stakeholders	`security.md` §3	`users` / RBAC Middleware	Security	Security test: Marketer blocked from `/approve`
`FR-018`	Anti-Self-Approval Enforcement	`ST-001`, `ST-002`	`security.md` §4	Domain Aggregate + DB Trigger	Compliance, SOX 404	Unit + DB test: Submitter blocked from self-review
`FR-019`	Decoupled Lifecycle Notifications	All Stakeholders	`failure-recovery.md` §3.2	`outbox_events` / Dispatcher	Reliability, Resilience	Chaos test: Outbox succeeds during FCM outage

3. Non-Functional Requirements Traceability Matrix (`NFR-001` to `NFR-017`)

NFR ID	CATEGORY	REQUIREMENT	TARGET METRIC / BOUND	IMPLEMENTATION MECHANISM	TRACEABLE DOCUMENTS
`NFR-001`	Consistency	Approval State Consistency	Zero split-brain states	Database-level Optimistic Concurrency Control on `(version, stage_index)`	`reliability.md` §3.3, `campaign-lifecycle.md` §5
`NFR-002`	Consistency	Version-Approval Binding	100% cryptographic binding	SHA-256 snapshot hash verification at scheduling gate	`security.md` §5.2, `revision-workflow.md` §3
`NFR-003`	Consistency	Concurrent Edit Safety	Zero lost updates	Version token check (`WHERE version = :expected`)	`reliability.md` §3.3, `failure-recovery.md` §3.3
`NFR-004`	Reliability	Operation Idempotency	100% duplicate-free execution	Client-generated UUID `Idempotency-Key` header with 24h TTL	`reliability.md` §2, `failure-recovery.md` §3.1
`NFR-005`	Reliability	Partial Failure Recovery	100% blast-radius isolation	Transactional Outbox pattern with exponential backoff & DLQ	`reliability.md` §3.1, `event-catalog.md` §4
`NFR-006`	Reliability	System Availability	99.9% uptime	Multi-AZ stateless API containers, PgBouncer, automated DB failover	`reliability.md` §4
`NFR-007`	Reliability	Duplicate Scheduling Guard	Zero duplicate dispatches	Atomic state transition `APPROVED -> SCHEDULED` on OCC	`campaign-lifecycle.md` §4.8
`NFR-008`	Security	User Authentication	Zero unauthenticated writes	OAuth 2.0 / OIDC JWT validation via asymmetric RS256	`security.md` §2
`NFR-009`	Security	Authorisation Enforcement	Zero unauthorized state actions	Server-side RBAC middleware evaluating role-to-stage permissions	`security.md` §3
`NFR-010`	Security	Self-Review Prevention	100% SoD enforcement	Tri-layer barrier: API Guard, Domain Invariant, Database Trigger	`security.md` §4
`NFR-011`	Security	Tenant Isolation	Zero cross-tenant data leakage	Logical tenant discriminator (`tenant_id`) & row-level security	`security.md` §2.1
`NFR-012`	Auditability	Immutable Audit Trail	Append-only (Zero update/delete)	PostgreSQL declarative monthly partitioned table, restricted DDL	`observability.md` §6, `database-schema.md` §2.6
`NFR-013`	Auditability	Audit Retention	12 months minimum (WORM ready)	Range partitioning by month; immutable cold-storage archival	`observability.md` §6
`NFR-014`	Performance	Request Response Time	Read $< 120\ text{ms}$; Write $< 250\ text{ms}$	Partial B-tree indexes, immutable snapshot Redis caching	`scalability.md` §2.2, §4.1
`NFR-015`	Scalability	Workload Scalability	5–50 concurrent users ($10\times$ headroom)	Stateless horizontal API pods, PgBouncer connection pooling	`scalability.md` §2.1, §3
`NFR-016`	Observability	Diagnostic Logging	100% structured JSON logs	Single-line JSON with correlation IDs, version, stage, actor, outcome	`observability.md` §3
`NFR-017`	Observability	Health Monitoring	Automated probe detection	`/health/liveness`, `/health/readiness` (DB & Outbox backlog checks)	`observability.md` §7.1

4. Architectural Decisions to Quality Attributes Mapping

ARCHITECTURAL DECISION	ADDRESSED PROBLEM / GOAL	DIRECT QUALITY ATTRIBUTE BENEFICIARY	REJECTED ALTERNATIVE & JUSTIFICATION
AD-01: Multi-Level Sequential Pipeline	Conflating creative, legal, and financial approvals violates SOX 404 & FINRA 2210.	**Security (`NFR-009`, `NFR-010`), Compliance, Reliability**	*Single-Gate Pool:* Fails regulatory audits. *Dynamic BPMN Engine:* **HIGH** failure overhead.
AD-02: Material Change Invalidation Engine	Post-approval drift where approved v1 copy is altered before release.	**Consistency (`NFR-002`), Governance (`FR-014`)**	*Any-Edit Invalidation:* Paralyzes marketing agility. *Silent Mutation:* Disastrous legal exposure.
AD-03: Immutable Full Snapshot Algebra (vN)	Delta diff reconstruction lag and state corruption on mobile devices.	**Performance (`NFR-014`), Auditability (`NFR-012`)**	*Incremental Event Sourcing Diffs:* Projection lag and reconstruction drift on mobile.
AD-04: Optimistic Concurrency Control (OCC)	Mobile connection drops holding database locks freeze reviewers.	**Reliability (`NFR-001`, `NFR-003`), Scalability**	*Pessimistic Row Locking (`SELECT FOR UPDATE`):* Deadlocks and connection pool starvation.
AD-05: Transactional Outbox Pattern	Downstream push notification (FCM) timeouts aborting business approvals.	**Reliability (`NFR-005`), Availability (`NFR-006`)**	*Synchronous RPC / Dual-Write:* **HIGH** latency, phantom alerts, partial failures.
AD-06: Tri-Layer Anti-Self-Approval Enforcement	Submitting marketers attempting self-authorization of ad campaigns.	**Security (`NFR-010`), Compliance (SOX 404)**	*Client-Only UI Hiding:* Easily bypassed via curl. *Configurable Policy:* Invites social engineering.

5. MAL4 Evaluation Rubric Coverage

```
pie title MAL4 Hackathon Scoring Weight Allocation
    "Architecture & Structure" : 60
    "Testing & Observability" : 15
    "Working User Journey (Mobile)" : 15
    "Architecture Defence & Trade-offs" : 10
```

MAL4 RUBRIC DIMENSION	WEIGHT	PRIMARY ARCHITECTURAL DOCUMENTS PROVING COMPLIANCE
Structure & Boundaries	**15%**	`domain-model.md`, `database-schema.md`, `user-journey-flow.mmd`
Data & State Management	**15%**	`campaign-lifecycle.md`, `revision-workflow.md`, `database-schema.md`
Testing & Observability	**15%**	`observability.md`, `traceability.md`, structured JSON logs, Prometheus metrics
Failure & Recovery Handling	**10%**	`reliability.md`, `failure-recovery.md`, Transactional Outbox, DLQ, Idempotency
Trade-offs & Adaptability	**10%**	Paired trade-offs across all sections, `trade-offs.md`
Problem Framing & Scope	**10%**	`problem-statement.md`, `stakeholders.md`
Working User Journey (Mobile)	**15%**	Mobile-first journey, structured field diffs, offline drafting, idempotency retry tokens
Architecture Defence	**10%**	Systematic defense of chosen approaches vs. rejected alternatives across all 01–07 documents